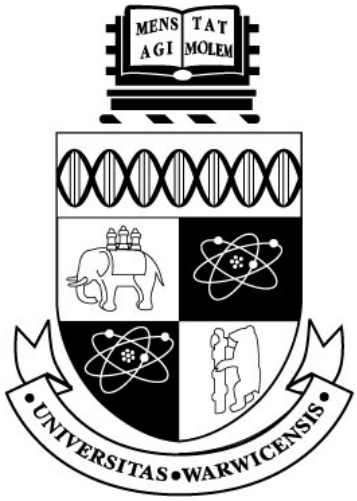
WARWICK
THE UNIVERSITY OF WARWICK



DISPLAY OPTIMISATION AND EXPERIMENTAL EVALUATION OF LARGE VISUAL BEHAVIOUR TREES IN GODOT 4.6

by

[AUTHOR NAME]

A thesis submitted in partial fulfilment

of the requirements for the degree of

MASTER OF SCIENCE IN GAMES ENGINEERING

UNIVERSITY OF WARWICK

WMG, University of Warwick

September 2026

Contents

List of figures

List of tables

Abstract

Visual behaviour trees are commonly used to author game-agent decisions, but traditional node–link editors become difficult to inspect as the number of nodes and runtime information increases. This dissertation designs and evaluates a Godot 4.6 editor plugin that combines a complete, executable behaviour-tree system with resettable display methods for large trees. The runtime supports ordered, reactive, random, parallel, repeating and timed control flow, Actor method calls, a typed blackboard, Decorators, and runtime inspection available only in the editor. The display experiment compares only six conditions: Baseline with the complete display, Compact Cards with reduced cards, Optimized Overview combining compact cards with low-detail semantic zoom, Optimized Search highlighting a fixed target, Subtree Focus showing only a specified branch and necessary context, and Context Collapse collapsing selected subtrees.

Evaluation first used automated tests and a playable 241-node behaviour tree to confirm that the experimental platform was correct. The six conditions were then evaluated through repeated paired measurements on deterministic trees ranging from 31 to 364 nodes and on three physically different screen sizes, examining card occupancy, visible context, target location and display-state reset. Results show that Compact Cards and Optimized Overview reduced card occupancy while retaining every node; Optimized Search reliably located and highlighted the target; and Subtree Focus and Context Collapse reduced irrelevant context currently displayed. All methods preserved node positions, tree structure and execution order in testing.

1 Introduction

1.1 Research Background

Modern game agents need to execute complex behavioural logic. For a small prototype, writing all decisions in a single procedural script is feasible; however, as states and interruption conditions increase, the control flow becomes increasingly difficult to inspect and modify. A behaviour tree (BT) represents decision logic as a hierarchy composed of control nodes and leaf nodes. Composite nodes determine how children are selected, while conditions and actions connect the abstract decision structure to game state and Actor code. High-level policies can be edited independently of movement, animation or combat methods, which is one of the principal values of behaviour trees in game development (Colledanchise and Ögren, 2018).

Traditional node–link trees can directly express parent–child topology, but rapidly consume space as their width and depth grow. Behaviour-tree cards may also display parameters, descriptions, Decorators, blackboard keys and execution states, making them larger than ordinary hierarchies that display only labels. When dozens or hundreds of cards are placed on a limited canvas, developers must frequently zoom, pan and search, and the current execution path may be submerged among inactive branches. This is not merely an aesthetic issue, because the node graph is the main tool through which a designer locates tasks, understands order, modifies conditions and diagnoses failures.

This project therefore uses a functionally complete Godot behaviour-tree plugin as a platform for research into large-tree display. While preserving upper and lower ports and the semantic rule that siblings execute from left to right, it adds independently switchable focus+context, complexity-management, navigation and runtime-explanation techniques. Each technique can be disabled independently, both to prevent a malfunctioning visual feature from destabilising the editor and to retain a reproducible baseline comparison condition.

1.2 Problem Definition and Research Gap

The fields of tree and graph visualisation have proposed many methods. Fisheye and focus+context views allocate more space to elements near the focus (Furnas, 1986; Lamping et al., 1995); overview+detail preserves a global reference view (Cockburn et al., 2008); multi-column layouts can improve browsing in trees with large fan-outs (Song et al., 2010);

incremental expansion and collapse can reduce graph complexity while attempting to preserve the user's mental map (Dogrusoz et al., 2018; Misue et al., 1995); and research into large layered graphs further shows that path-task efficiency depends on the particular representation and task (Bae and Watson, 2011).

However, this research does not directly answer how a game behaviour-tree editor should combine these mechanisms. Behaviour trees differ from ordinary hierarchies such as file directories: sibling order has execution semantics; leaf nodes invoke code; Decorators may prevent a branch from running; and graph state changes on every runtime Tick. A usable solution must preserve execution order, support editing and undo, expose blackboard state, and explain failures without adding a debugging overlay to the game view. Existing research can guide individual design choices, but an integrated, executable and reproducibly evaluated Godot implementation is still required.

1.3 Research Aim and Question

This dissertation aims to design and evaluate composable large-tree display optimisations on a functionally validated Godot 4.6 visual behaviour-tree platform capable of driving real NPCs. The research question is: across different behaviour-tree node scales and physical screen sizes, can the composable display optimisations proposed by this project improve the measurable display and editing-interaction performance of large behaviour trees compared with a traditional baseline display, while preserving behaviour-tree structure and execution semantics?

1.4 Research Objectives and Success Criteria

The project work is divided into platform construction and research evaluation. Platform construction comprises implementing serialisable resources for trees, nodes, Decorators and typed blackboards; implementing the SUCCESS, FAILURE and RUNNING semantics of Root, Sequence, Selector, Reactive Selector, Random Selector, Parallel, Repeat, Wait, Action and Condition; allowing a reusable component to assign a tree and an Actor to an NPC, with leaf nodes calling Actor methods; providing creation, connection, disconnection, dragging, typed editing, validation, saving, loading, undo and redo; and displaying paths, node states, the blackboard and failure reasons only in the Godot editor.

Research evaluation provides independent persistent switches and safe reset paths for large-tree display methods, and uses raw data to compare geometric presentation, target location and context retention across different node scales, physical screen sizes and conditions before and after optimisation. Before conducting the display-optimisation experiments, the fundamental behaviour-tree plugin functions must first be confirmed as correct: each node type must execute as expected; an edited behaviour tree must save and reopen correctly; undo and redo must not damage tree content; and the complex enemy in the playable scene must actually be controlled by the 241-node behaviour tree. Display op-

timisations must be compared with the original display in a real rendering environment. After an optimisation is disabled, the editor must return to its original display state and must not change the behaviour-tree structure, node positions or execution order. If testing produces an error, crash, memory leak or abnormal exit, the test is judged to have failed even if other checks report success.

2 Literature Review

2.1 Behaviour Trees as Executable Hierarchies

A behaviour tree repeatedly evaluates a rooted hierarchy and propagates one of three states: SUCCESS, FAILURE or RUNNING. A Sequence normally stops at the first child that fails or is running; a Selector stops at the first child that succeeds or is running; and leaf nodes read state or perform actions. This small state interface enables complex logic to be composed and interruption policies to be expressed explicitly (Colledanchise and Ögren, 2018).

The three-state interface appears simple, but it contains important implementation choices. A running Sequence can remember its current index, whereas a Reactive Selector must recheck high-priority conditions and pre-empt a lower-priority branch. A Random Selector should retain the same choice while its child is running; Parallel requires success and failure thresholds; and Repeat and Wait need to clear their memory when restarted or interrupted. Editor presentation and execution semantics therefore cannot be separated completely: child order, Decorator ownership and node identity all affect execution and must be serialised correctly.

A blackboard provides shared key–value state between perception, conditions and actions. Unreal Engine's workflow demonstrates the value of combining a blackboard, Decorators and runtime observation (Epic Games, n.d.). This project does not, however, need to reproduce every capability of a commercial system. A reasonable baseline is a system that can express meaningful decisions, call project Actor methods, validate blackboard keys, and explain why a branch ran or was rejected.

2.2 Node–Link Trees and Scalability

Node–link diagrams are appropriate for behaviour trees because edges can directly represent ancestry and sibling groups. Classical tidy-tree algorithms seek layouts with consistent levels, non-overlapping subtrees and relative compactness (Reingold and Tilford, 1981). SpaceTree further combined a node–link structure with dynamic layout, zooming and selective expansion; its evaluation showed that representation and interaction jointly affect topology-understanding and revisit tasks (Plaisant et al., 2002).

The problem is scale. Cards occupy area, and explicit edges become denser as width and depth increase. Song et al. compared traditional, list and multi-column views, finding that the multi-column interface was faster in their browsing and understanding tasks for

hierarchies with large fan-outs and was preferred overall (Song et al., 2010). This result is relevant to Selectors or Sequences containing many alternative behaviours, but it cannot be transferred directly: children in that study were ordered alphabetically, whereas horizontal order in a behaviour tree may determine priority. A multi-column behaviour tree must therefore display order explicitly and must not silently modify the resource during layout.

Bae and Watson proposed the Quilts representation for large layered graphs (Bae and Watson, 2011). Its value is not necessarily that a behaviour tree should be converted completely into a matrix, but that path tasks may be better supported by a supplementary representation. This project therefore retains an editable node graph while adding a compact textual path and strong active-path encoding for runtime path-location tasks.

2.3 Focus, Context and Overview

Furnas formalised generalised fisheye views with a degree-of-interest function: an element's prior importance and its distance from the focus are combined to determine its display prominence (Furnas, 1986). Lamping et al. used hyperbolic geometry to display a large hierarchy in a bounded region, magnifying the focus while compressing distant context (Lamping et al., 1995). A behaviour-tree editor can use this idea to make the node under the pointer readable while retaining surrounding structure.

Focus+context is not inherently superior to other methods. Distortion can move targets and increase pointing difficulty, while changes in scale can also damage the mental map. Cockburn et al. distinguish overview+detail, zooming and focus+context, and note that effectiveness depends on task, implementation and switching cost (Cockburn et al., 2008). This project therefore limits maximum fisheye magnification to 1.20, changes the internal card representation rather than applying an unstable whole-graph transform, compensates position around the node centre, and restores all geometry when the feature is disabled. It also provides an independent fixed minimap so that the two types of technique can be compared or combined.

Semantic zoom differs from geometric zoom. Geometric zoom changes physical size, whereas semantic zoom changes displayed attributes. A low-detail card retains type colour, icon and short title; a high-detail card displays the method, parameters, description and Decorators. It trades information completeness for density without changing execution-graph geometry. An earlier version of the plugin coupled the mouse wheel to temporary node scaling, causing nodes to shrink and then grow while scrolling; separating information visibility from graph geometry eliminated this problem.

2.4 Complexity Management and the Mental Map

When all nodes do not need to be viewed at once, expand–collapse and hide–show operations can reduce complexity. Dogrusoz et al. proposed incremental methods for managing the complexity of large networks, including expansion, collapse and layout updates

(Dogrusoz et al., 2018). This research is consistent with the mental-map principle: if one edit causes many unrelated nodes to move, users must relearn their positions (Misue et al., 1995). Behaviour-tree collapse should retain the collapsed root, report hidden content, and avoid moving unrelated branches where possible.

Subtree collapse and subtree focus address different problems. Collapse replaces descendants with a summary while retaining the rest of the tree; focus displays only the target branch and necessary context. Both may hide a running node, so the plugin provides hidden-node counts, two-level summaries, path indicators, and explicit expand/focus controls instead of removing view information without notice.

Search highlighting and inactive-branch dimming are softer forms of complexity management: geometry remains unchanged and only the prominence of irrelevant content is reduced. This follows Shneiderman's principle of "overview first, zoom and filter, then details on demand" (Shneiderman, 1996). Shape/icon encoding and a colourblind-safe palette provide redundant channels so that node types are not expressed by colour alone.

2.5 Runtime Explanation and Failure Localisation

Live debugging adds a temporal dimension to a static hierarchy. Highlighting only the leaf node cannot explain the path by which it was reached; highlighting all historical nodes, by contrast, confuses the current state. This project records an ordered Root-to-leaf chain, per-node states and the current failure explanation. Inactive branches can be dimmed, while an independent path summary remains readable when the graph is reduced.

Decorators make failure explanation more important. An Action itself may be correct but never execute because a blackboard comparison, cooldown or time limit prevents it. Displaying a Decorator summary on its owner node and attaching a failure reason connects runtime evidence back to the authoring representation; the live blackboard further displays each Key, runtime type, value and Schema state. This reflects the task-oriented principle from layered-graph research: during debugging, the user is often asking "why did this branch not run?" rather than "what is the complete topology?"

3 Research Methodology

3.1 Research Strategy

This study first implements a Godot 4.6 plugin capable of authoring and running behaviour trees correctly, and then uses that plugin to test display optimisations for large behaviour trees. The research focus is not whether behaviour trees are suitable for making games, but whether these display optimisations can improve measurable interface performance across different node counts and screen sizes.

3.2 Experimental Factors and Datasets

The experiment uses five automatically generated behaviour trees containing 31, 61, 121, 241 and 364 resource nodes. They cover a small behaviour tree, a behaviour tree beyond the approximately 50-node problem raised by the supervisor, and larger stress-test cases. The five trees use the same generation rules, node-type proportions and Decorator proportions, with scale as their principal difference. Decorators are attached to other nodes and are not displayed as separate canvas cards, so the five trees are displayed as 26, 51, 101, 202 and 305 cards respectively.

The project also contains a 241-node behaviour tree that directly controls a game enemy. It includes healing, melee attack, ranged attack, jumping, climbing, pursuit, search, retreat, patrol and idle behaviours. This tree is used only to confirm that the plugin can correctly control a complex enemy; it is not used as a separate display-optimisation experiment.

This dissertation compares only the six display conditions listed in Table 3.1. Before every trial, the same initial state is restored: all nodes are expanded, search is cleared, focus is cancelled, zoom is set to 100%, and the display optimisations involved in the experiment are disabled. Only the operations specified for the current condition are then applied.

Table 3.1: The six display conditions and the operations performed in the experiment.

Condition	Operation performed in the experiment	Interface change and main measurement purpose
Baseline	Disable the display optimisations involved in the experiment and show full cards and every branch.	Provide the original display as the comparison baseline for the other conditions.
Compact Cards	Enable only compact cards. Every node remains visible, but cards become smaller and retain only the type and short title.	Measure how much total card area is reduced without hiding nodes.
Optimized Overview	Enable compact cards and low-detail semantic zoom together, and set canvas zoom to 50%.	Retain every node but reduce the information on each card in order to view the structure of the complete tree.
Optimized Search	In the Overview state, search for a predefined node with a unique name.	Highlight the target, dim other nodes, and check whether the target enters the current viewport.
Subtree Focus	Select a predefined branch as the focus target.	Display only the target branch and necessary context, reducing irrelevant nodes in the current view.
Context Collapse	Retain the path to a fixed target and collapse selected subtrees outside that path.	Retain each collapsed root, replace its descendants with a summary, and reduce the number of simultaneously displayed nodes.

The six conditions change only card size, information content, prominence or the current display range. Even when Collapse is used, the plugin saves only the collapsed state and does not change parent–child relationships or execution order.

The experiment records the number of displayed cards, total card area, the amount of information retained on cards, the number of non-target nodes dimmed by Search, whether the search target enters the viewport, and how much current context is reduced

by Focus and Collapse. Other display functions do not participate in the condition comparison in this dissertation. These data describe only changes in the interface display and cannot directly demonstrate that users understand behaviour trees more easily.

3.3 Controlled Multiscale Display Experiment

The controlled experiment uses an NVIDIA GeForce RTX 5070 Laptop GPU, the Godot 4.6 OpenGL Compatibility renderer, and a fixed 1600×900 test canvas. The experiment contains five node scales and six display conditions. Every scale–condition combination is formally tested 30 times, producing $5 \times 6 \times 30 = 900$ observations in total.

Each group is run three times before formal testing, but these results are not recorded. This warm-up reduces variation caused by first loading and rendering. Formal testing rotates the order of tree scales and display conditions so that one condition is not always executed first or last. Every test starts from the same state: expand every node, clear search, cancel focus, use 100% zoom, and then apply only the condition currently being tested. Search and Focus use predefined fixed targets to prevent manual selection from affecting the results.

Minimum acceptance criteria were defined before the experiment. Baseline must display every card, and display optimisations must not change saved positions or execution order. Compact must reduce total card area by at least 40%, and Overview by at least 70%; from 61 nodes upwards, Search must dim at least 95% of non-target cards; and from 121 nodes upwards, Focus and Collapse must reduce displayed cards by at least 70%.

3.4 Physical Screen-Size Experiment

The second experiment compares only physical screen size, not resolution. Windows Extended Display Identification Data (EDID) reported physical dimensions of 34×22 cm, 60×33 cm and 70×39 cm for the three screens. The experiment converts every centimetre to 35 logical units, so the three screens correspond to test canvases of 1190×770, 2100×1155 and 2450×1365. Resolution, refresh rate and operating-system scaling are recorded only as device metadata and do not participate in the comparison.

All five behaviour trees and all six display conditions are tested on each screen. Every screen–scale–condition combination is warmed up twice and then formally recorded three times, producing $3 \times 5 \times 6 \times 3 = 270$ observations in total.

This experiment principally records the proportion of cards that fit on each screen, the ratio of behaviour-tree area to screen area, card area, whether the search target enters the viewport, and the context reduced by Focus and Collapse. It also checks that node positions and execution order remain unchanged.

3.5 Plugin and Playable-Game Validation

Before the display experiments, the plugin itself is first confirmed to operate correctly. Automated tests are divided into four categories: resource and runtime, editor interface, basic game, and complex arena. They cover node execution, Decorators, Actor method calls, creation and connection, saving and reloading, undo and redo, and basic interactions between the player and enemies.

The playable scene also confirms that the 241-node behaviour tree can actually control enemy detection, defence, melee attack, ranged attack, jumping, climbing, search, retreat and healing. This part shows only that the plugin is functionally correct and can be used in a non-trivial game scenario; it is not an independent experiment on the effect of display optimisation.

If a test produces a non-zero exit code, failure message, script error, memory leak, invalid access or program crash, the entire test group is judged to have failed. Only warnings deliberately triggered by a test and producing the expected result are accepted.

4 System Design and Implementation

4.1 Architecture and Project Boundary

The system is divided into editor, resource, runtime and test components, as shown in Figure 4.1. The game executes behaviour trees through resource files and runtime components and does not depend on the editor interface. Live Debug also appears only in the Godot editor and is not added to the final game view. The editor interface is integrated through Godot's EditorPlugin extension mechanism, and its node canvas is based on GraphEdit (Godot Engine contributors, n.d.-a,n).

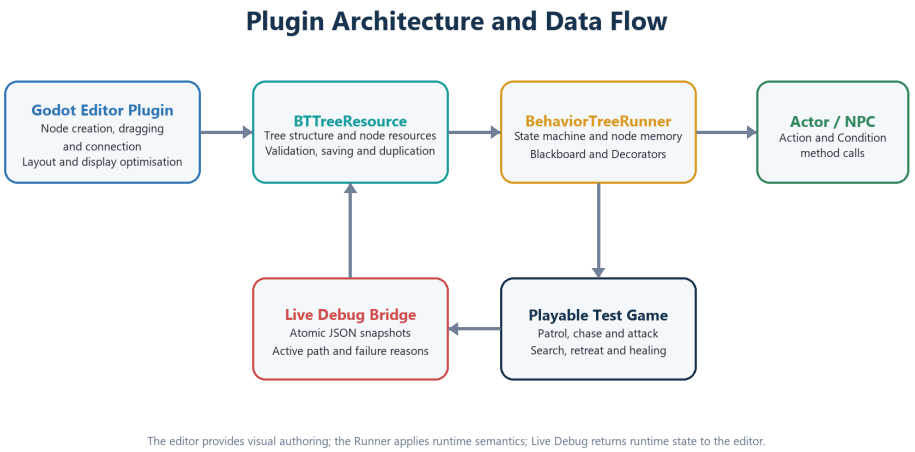


Figure 4.1: Plugin architecture and data flow between the Godot editor, behaviour-tree resource, Runner and Actor.

`BTTreeResource` stores the complete tree, while `BTNodeResource` stores the type, parameters, position and parent relationship of an individual node. `BehaviorTreeRunner` executes the behaviour tree, and `BehaviorTreeComponent` connects the tree to an Actor in the scene. The main editor interface is responsible for the canvas, property editing, blackboard, history and Live Debug. This division allows resources, execution logic and the display interface to be tested separately.

4.2 Resource Model and Validation

Ordinary nodes record their parent in `parent_id`, while Decorators use a separate `decorator_parent_id`. A Decorator is therefore not incorrectly executed as an ordinary

child. Sibling nodes are sorted by horizontal position on the canvas, so the visible order from left to right is also the execution order. Node positions and this order are both retained when the resource is saved.

Before saving, the plugin checks that a Root exists, node IDs are not duplicated, parents are valid, Decorators are attached correctly, and node parameters are complete. For example, a Root cannot have a parent, while an Action or Condition cannot own ordinary children. Editor saving and automated testing use the same validation rules.

The blackboard Schema can declare Bool, Int, Float, String and `Vector2` types and can set a default value for each key. Strict mode reports undeclared or invalid keys. The Inspector provides a key picker while retaining free text input. Schema changes are also included in undo and redo history.

4.3 Runtime Semantics

Every behaviour-tree update returns SUCCESS, FAILURE or RUNNING. Nodes that must continue across frames save progress by ID; these records are cleared when the tree is restarted, replaced or a branch is interrupted. Table 4.1 summarises the behaviour of each node type.

Table 4.1: Implemented runtime node semantics.

Node	Behaviour
Root	Executes its single ordinary child.
Sequence	Executes children in order, stops on failure or running, and remembers the running index.
Selector	Executes children in order, stops on success or running, and remembers the running index.
Reactive Selector	Rechecks from the highest priority on every Tick and can interrupt a lower-priority branch.
Random Selector	Uses a seedable random choice and retains the same child while running.
Parallel	Ticks multiple children and applies success and failure thresholds.
Repeat	Repeats a fixed number of times or indefinitely and preserves the count across Ticks.
Wait	Accumulates delta and succeeds when the duration is reached.
Condition	Calls an Actor predicate or compares a blackboard value.
Action	Calls an Actor method and converts a Bool or status into a tree state.

Action and Actor-mode Condition nodes call Actor methods through a common in-

terface. If a method does not exist, the node returns failure and displays the reason instead of crashing the game. Blackboard conditions support key existence, truth, equality, inequality and numeric comparison.

Decorators change a condition or result before or after their owner node executes. Implemented behaviours include blackboard comparison, cooldown, time limit, result inversion, forced result and repetition. Resetting a subtree clears the saved execution progress of both Decorators and child nodes.

4.4 Editor Authoring Workflow

The plugin is displayed in the bottom panel of the Godot editor. Users right-click the canvas to create a node and do not need to face a row of creation buttons in the menu bar. Nodes can be moved, connected, disconnected, duplicated and deleted. The behaviour-tree selector displays only resource names, so users do not need to view or enter complete file paths manually. Creation, deletion, movement, connections, parameter changes, Schema changes and collapsed state all support undo and redo.

The Inspector uses controls corresponding to field types. For example, an Action directly enters a method name, while a Condition selects Actor or blackboard mode and sets a key, comparison and value. Random, Parallel, Repeat, Wait and Decorator nodes also have their own settings. Advanced JSON is retained only as a compatibility entry and is not required for normal operation.

The plugin uses connection ports above and below nodes, whereas Godot's default connections are mainly designed for left and right ports. The plugin therefore draws the final connections itself while using Godot's preview when a connection is dragged. Custom connections can still be disconnected with a right click and support undo and redo. Disabling custom display restores Godot's native connections.

4.5 Live Debug and Blackboard Inspection

At runtime, the current Actor, behaviour tree, active node, execution path, failure reason and blackboard values are written to a debug file. The old version is replaced only after the file has been completely written, preventing the editor from reading incomplete content. If one read fails, the editor retains the previous complete state and retries on the next update.

The editor displays debugging information only for the current behaviour tree. The active path uses a prominent border, inactive branches can be dimmed, failed nodes display a reason, and the blackboard panel displays current keys and values. Long paths are placed in a scrollable area to prevent them from covering the main canvas. None of this debugging content appears in the game view. An example is shown in Figure 4.2.

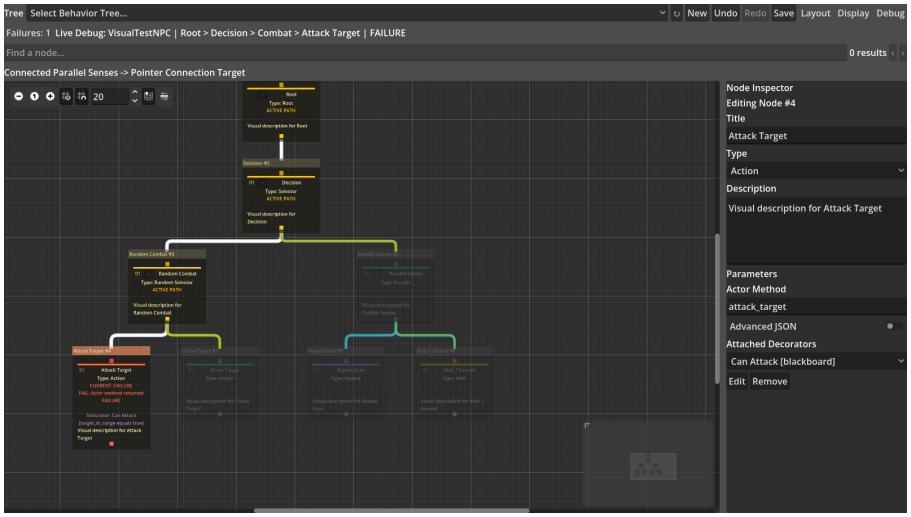


Figure 4.2: Editor-only display of the active path and failure reason.

4.6 Large-Tree Display Mechanisms

The problem of a large behaviour tree comes not only from node count, but also from the simultaneous growth of card information, connection density, navigation distance and runtime state. The plugin therefore provides display functions in four areas: information density, focus and structural reduction, navigation and overview, and connection and runtime representation.

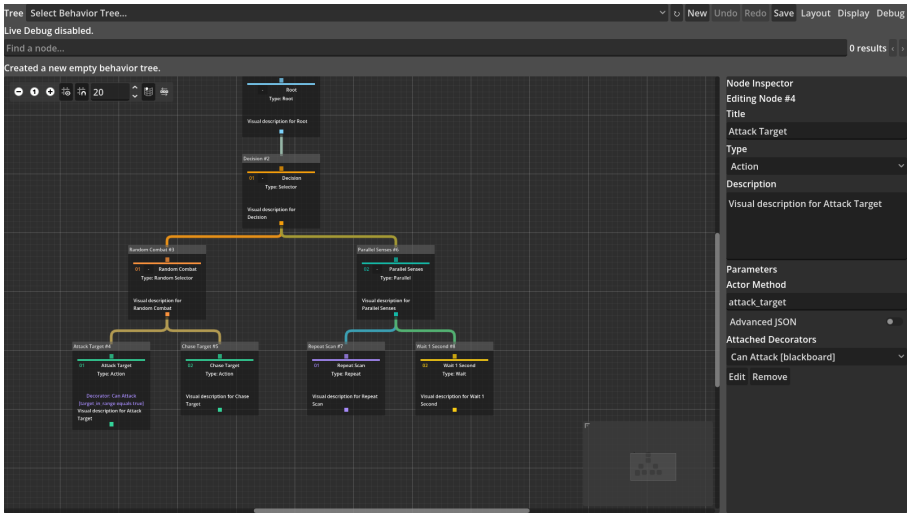
4.6.1 Information-Density Control

Compact reduces card size without hiding nodes. Cards retain the node type and short title, while parameters, descriptions and Decorator summaries can be displayed when needed. Semantic Zoom changes the information level according to the canvas zoom ratio: close views display complete fields, medium views retain titles and key parameters, and distant views retain only the information required to identify nodes. Neither function changes the behaviour-tree resource.

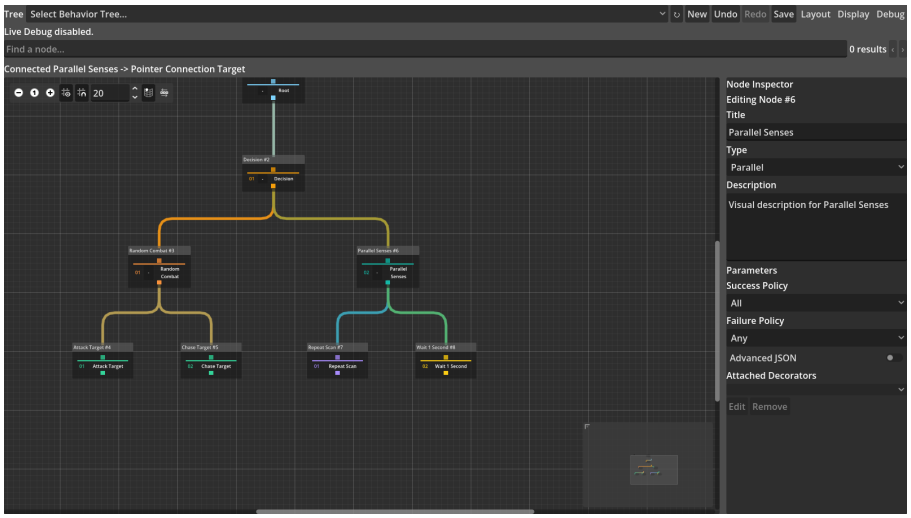
Node types are also redundantly encoded with colour, icons and shapes. Even when colours are difficult to distinguish, users can identify the Root, composite nodes, tasks and Decorators through icons and outlines. The plugin also provides a colourblind-safe palette, reducing reliance on a single colour channel. The two information-density states are shown in Figure 4.3.

4.6.2 Focus and Structural Reduction

Fisheye Focus+Context enlarges cards near the pointer while retaining surrounding nodes as positional references. Maximum magnification is limited to 1.20 times, and card positions are compensated around their own centres to prevent the overall layout from drifting



(a) Baseline

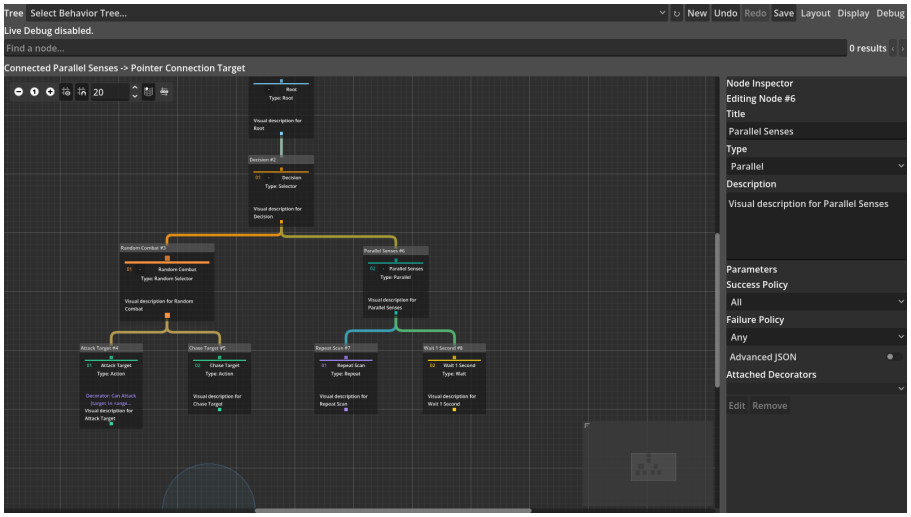


(b) Compact cards

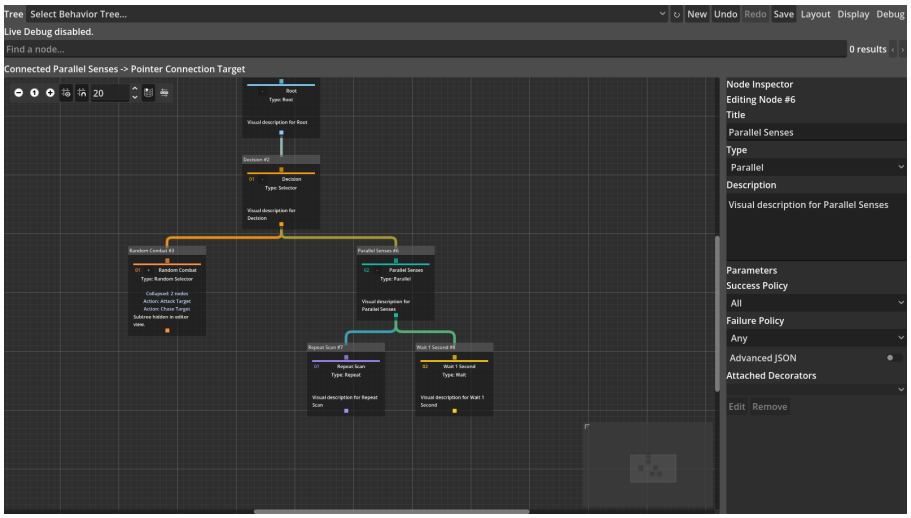
Figure 4.3: Full-detail Baseline and Compact Cards for the same behaviour tree.

continuously after magnification. When fisheye is disabled, cards return to their original size and position.

Search supports the title, type, description and parameters of Action, Condition and Decorator nodes, and provides previous, next and clear operations. Matching nodes remain prominent, other nodes are dimmed, and the selected target can be moved into the current viewport. Subtree Focus displays only the specified branch and necessary context; Context Collapse retains the collapsed root and a summary of hidden nodes but temporarily does not draw descendants. After expansion, the original node positions and parent–child relationships are still used. Fisheye and collapse examples are shown in Figure 4.4.



(a) Fisheye focus and context



(b) Subtree collapse and summary

Figure 4.4: Two implementations of local magnification and structural reduction.

4.6.3 Navigation and Overview

The enhanced minimap uses a fixed area to display a reduced structure of the complete tree and marks the current viewport position. Fit moves the current graph into a visible range, Focus locates the selected node, and Show All cancels a partial display and returns to the complete tree. Breadcrumb displays the hierarchical path from the Root to the current node, while the path summary states the current execution or selection path in text, allowing the current location to remain identifiable when cards are reduced.

Search results can be traversed with buttons or the keyboard. Multi-column layout separates branches with many children into multiple columns to avoid unlimited horizontal expansion on one level; stable layout attempts to retain the positions of unmodified branches and reduce widespread movement caused by a single edit. Automatic arrangement still preserves the left-to-right execution order of siblings. The related navigation

controls are shown in Figure 4.5.

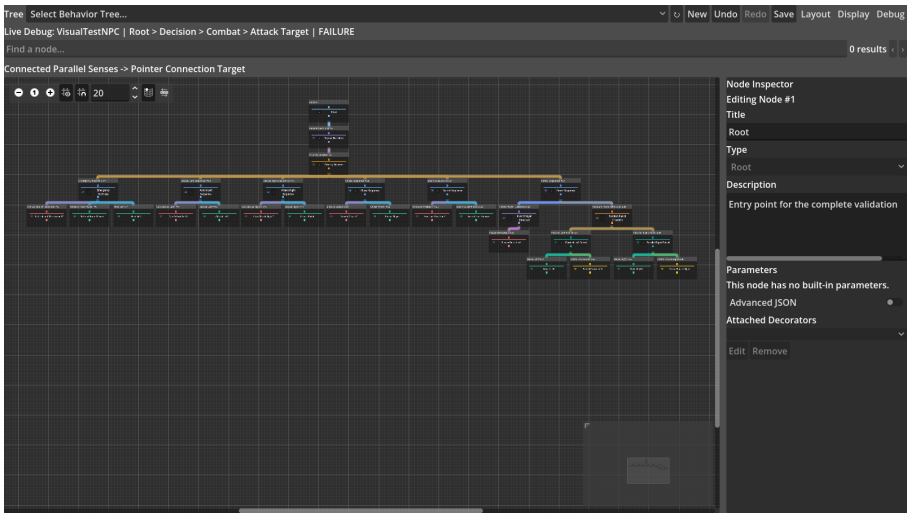


Figure 4.5: Minimap, path summary, navigation controls and automatic layout in a complex tree.

4.6.4 Connection and Runtime Representation

The plugin uses an output port below a node and an input port above it to express a parent–child relationship. Final connections are drawn by the plugin, while the drag preview continues to use Godot GraphEdit input handling. Orthogonal connections use horizontal and vertical segments to reduce curve crossings; edge bundling first gives connections with a common parent a shared trunk and then branches towards each child. Both methods retain connection clicking, disconnection, undo and redo.

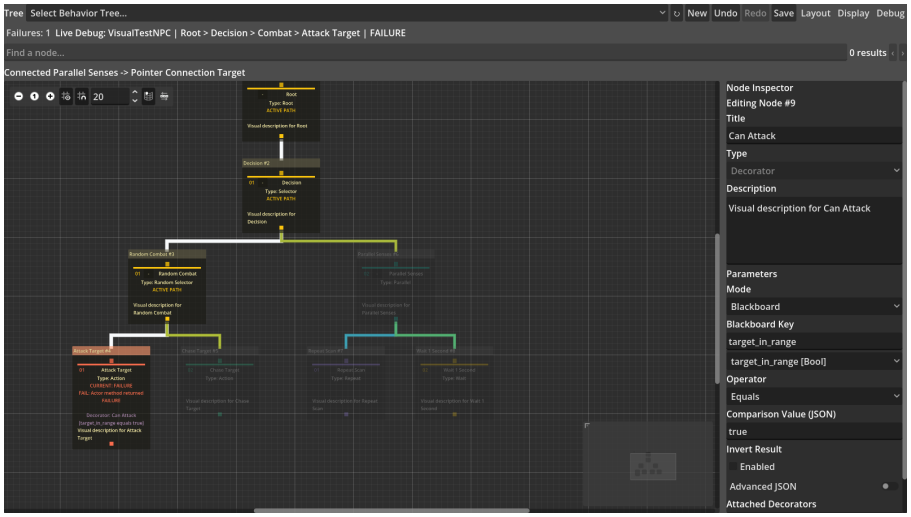
The runtime representation displays the active path, current node, success or failure state, inactive-branch dimming and failure reason on the same canvas. Path colour propagates continuously along parent–child relationships, and a Decorator failure is displayed near its owner node. Users can therefore see both the static structure and the branch actually traversed by the current Tick. Connection and runtime-state examples are shown in Figure 4.6.

4.7 Physical Screen Experiment Infrastructure

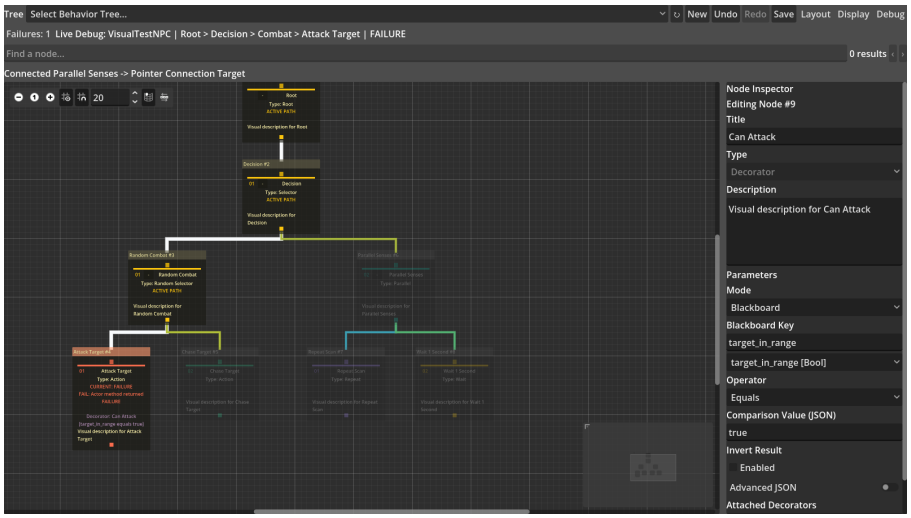
The screen-testing tool reads physical width and height from EDID and records them separately from resolution. For a screen of width W_{cm} and height H_{cm}, the test canvas is

$$W_c = 35W_{\text{cm}}, \qquad H_c = 35H_{\text{cm}},$$

where W_c and H_c are the logical width and height of the canvas. Every screen uses 35 logical units per centimetre, so the experiment compares physical display area rather than the pixel density of different devices.



(a) Orthogonal connections and active path



(b) Edge bundling and failure annotation

Figure 4.6: Connection organisation and runtime-state representation.

The test window moves to the specified screen and then loads the same behaviour tree and display condition. Every test saves device information and raw CSV data. When a new screen is connected in the future, its data are written to a new Session without overwriting existing results.

4.8 Demonstration Game

The complex two-dimensional arena confirms that the behaviour tree can control actual game enemies. The player can move, jump, climb, perform a melee attack, dash, heal and become briefly invisible. Three guards share the same 241-node behaviour tree. According to distance, health, obstacles and player state, guards select defence, melee attack, ranged attack, jumping, climbing, pursuit, search, retreat, healing, patrol or idle behaviour.

An enemy detects the player within 330 pixels and abandons the target only when the distance exceeds 460 pixels. Using two different distances prevents the enemy from switching state repeatedly near a boundary. The game uses simple graphics so that development remains focused on behavioural logic and test stability.

5 Results and Analysis

5.1 Plugin and Playable-Game Validation

Plugin functionality testing contained 450 checks and all passed, as shown in Table 5.1. The logs contained no script error, memory leak, invalid access or program crash. The subsequent display experiments therefore used a functionally correct behaviour-tree plugin.

Table 5.1: Final core automated regression.

Suite	Passed	Total	Principal coverage
Resource and runtime	153	153	Structure, node semantics, Decorators, Actors, Schema, debug bridge and execution reset after resource modification
Editor GUI	250	250	Simplified menus, context creation, graph editing, history, Schema and display-condition reset
Basic game	13	13	Three enemies, movement, damage, patrol, death/restart and lifecycle
Complex arena	34	34	Detection, reactive defence, melee/ranged attack, jumping, climbing, search, retreat and recovery
Total	450	450	All functionality and game assertions passed

The playable scene also passed testing. Depending on the situation, enemies pursue, perform melee attacks, defend, perform ranged attacks, jump over obstacles, climb ladders, search for the player, retreat and heal. Actor methods are responsible for actual movement and damage, while the 241-node behaviour tree selects behaviours, determines execution order and interrupts lower-priority behaviours. This part demonstrates only that the plugin can correctly control a complex enemy; it is not an independent display-optimisation experiment.

5.2 Controlled Multiscale Display Results

The controlled experiment produced the planned 900 observations and every predefined check passed. Table 5.2 presents the principal results for the five tree scales. Compact did not hide nodes but reduced total card area by 61.09–61.35%. Optimized Overview likewise retained every node and reduced total card area by 90.27–90.34%. This shows that both methods can reduce screen occupancy, but it does not show that users complete tasks faster by an equivalent proportion.

Table 5.2: Controlled display measurements for five resource scales; reductions are relative to Baseline.

Measure	31	61	121	241	364
Canvas cards	26	51	101	202	305
Compact area reduction	61.35%	61.30%	61.30%	61.16%	61.09%
Overview area reduction	90.34%	90.33%	90.33%	90.29%	90.27%
Search dimming	96.15%	98.04%	99.01%	99.50%	99.67%
Focus card reduction	76.92%	86.27%	87.13%	91.58%	85.90%
Collapse card reduction	69.23%	78.43%	89.11%	93.07%	96.39%

Search highlights the unique target and moves it into the viewport. The proportion of dimmed non-target nodes increased from 96.15% for the 31-node tree to 99.67% for the 364-node tree. Focus and Collapse reduce the context currently displayed but do not delete resource nodes. The Focus reduction did not continue increasing for the 364-node tree because the fixed target owns a larger local branch in that tree. Every condition preserved node positions and left-to-right execution order. The reductions across scales are shown in Figure 5.1.

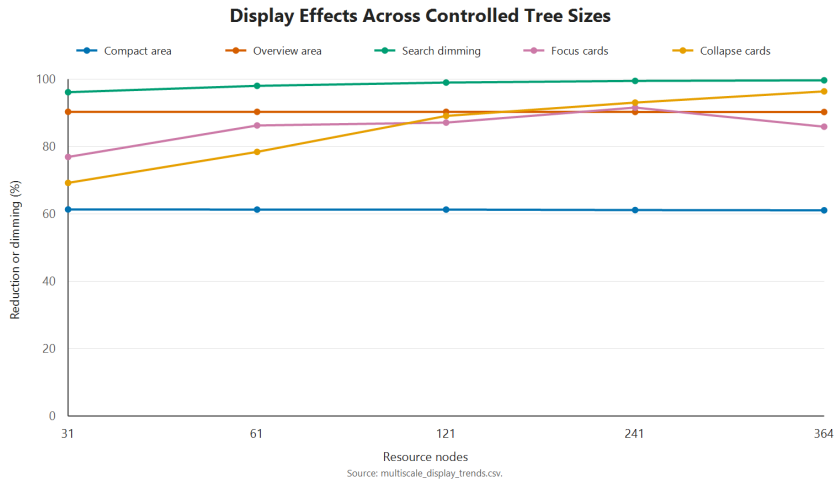


Figure 5.1: Reductions in density, salience and context across the five controlled scales.

5.3 Physical Screen-Size Results

All 270 tests on the three screens completed without failure. Table 5.3 lists the results for the 241- and 364-node trees. “Coverage” is the proportion of card centres that fit within the canvas; “graph/screen” is the number of times the complete behaviour-tree area exceeds the test-screen area.

Table 5.3: Complex-tree coverage on three physical screen sizes.

Screen	Surface (cm)	Canvas	241 coverage	364 coverage	241 graph/screen	364 graph/screen
BOE0CD1, 15.94 inches	34×22	1190×770	18.32%	12.13%	116.38x	184.13x
H27T22S, 26.96 inches	60×33	2100×1155	38.61%	25.57%	43.96x	69.56x
U32G3X, 31.55 inches	70×39	2450×1365	46.53%	30.82%	31.89x	50.45x

When screen size increased from 15.94 inches to 31.55 inches, Baseline coverage for the 241-node tree increased from 18.32% to 46.53%, while coverage for the 364-node tree increased from 12.13% to 30.82%. Coverage for both trees became 2.54 times the original value. The complete-tree area relative to the screen area decreased by 72.60%. This shows that a physically larger screen can display more context, but it does not demonstrate that developers necessarily understand that content more easily.

The relative effects of the display optimisations were broadly consistent across the three screens. Compact area reduction remained between 61.09% and 61.35%, while Overview remained between 90.27% and 90.34%. Search moved the target into the viewport in all 15 screen–scale combinations. Every condition also preserved node positions and execution order.

Structural reduction had a more visible effect on the small screen. For the 241-node tree, Subtree Focus coverage across the three screens was 82.35%, 88.24% and 88.24%, while Context Collapse coverage was 50.00%, 64.29% and 64.29%. For the 364-node tree, Subtree Focus coverage was 86.05%, 95.35% and 95.35%, while Context Collapse coverage was 36.36%, 54.55% and 54.55%. Here, coverage means the proportion of cards retained by the current condition that enter the canvas; it does not mean that the same proportion of the complete behaviour tree is simultaneously visible.

6 Discussion

6.1 Functional Correctness of the Experimental Platform

The complete plugin is the platform for the display-optimisation experiment, not a separate object of study. It can save and execute behaviour trees and supports Actors, composite nodes, conditions, actions, Decorators, the blackboard and Live Debug. The experiment therefore uses a behaviour tree that can actually execute, rather than a static node graph drawn only on a canvas.

All 450 checks across resource and runtime, editor interface, basic game and complex arena passed. The 241-node behaviour tree controls enemy detection, defence, melee attack, ranged attack, jumping, climbing, search, retreat and healing. These results show that the plugin provides a functionally correct test platform with actual game behaviour for the display-optimisation experiment.

6.2 Evaluation of Display-Optimisation Effects

6.2.1 Optimisation Effects across Different Node Scales

Compact retained every card at all five scales while reducing total card area by 61.09–61.35%. The reduction proportions are very similar across scales, showing that the effect of compact cards does not weaken substantially as node count increases. It is suitable for reducing the space occupied by individual cards when every node still needs to be viewed.

Optimized Overview likewise retained every card, with a stable area reduction of 90.27–90.34%. Compared with Compact, it further reduces information inside each card and is therefore more suitable for viewing the overall branch structure of a large behaviour tree. The results changed very little from 31 to 364 nodes, showing that this combination retains a similar improvement in information density across different tree scales.

The proportion of non-target cards dimmed by Optimized Search increased from 96.15% for the 31-node tree to 99.67% for the 364-node tree. As the behaviour tree becomes larger, the view contains more non-target nodes, so Search produces a more visible improvement in target prominence. The fixed target entered the viewport at every scale, showing that Search can reliably locate a known node in a large tree.

Subtree Focus reduced displayed cards by 76.92–91.58% across the five scales, with a

reduction of 85.90% for the 364-node tree. This proportion is determined by the size of the target branch itself, so it does not increase strictly with total node count. Context Collapse card reduction increased from 69.23% to 96.39%, showing that larger trees contain more non-target branches that can be collapsed. Both methods reduce irrelevant context in the current view, but Focus addresses a specified branch while Collapse retains more positional information about the overall structure.

Across the five scales, the area reductions from Compact and Overview remained stable, the target-highlighting effect of Search increased with scale, and Focus and Collapse directly reduced the context displayed at one time. These results show that the display optimisations already have an effect on small trees and are more apparent on the larger 121-, 241- and 364-node trees.

6.2.2 Optimisation Effects across Different Physical Screen Sizes

The screen-size experiment shows that physical display area directly affects the context that a large tree can present. When screen size increased from 15.94 inches to 31.55 inches, Baseline coverage for the 241-node tree increased from 18.32% to 46.53%, while coverage for the 364-node tree increased from 12.13% to 30.82%. Coverage for both complex trees became 2.54 times the small-screen value, showing that a larger physical screen can accommodate more nodes at once.

Different functions did not respond to screen size in the same way. Compact card-area reduction remained between 61.09% and 61.35% on all three screens, while Overview remained between 90.27% and 90.34%. Their relative effects were almost identical on the small and large screens, so they provide an information-density improvement that is independent of screen size and do not further amplify the advantage of a large screen. Because the 241- and 364-node trees had already reached the 0.10 minimum zoom, these methods substantially reduced card size and information content but did not allow the small screen to display more nodes from the complete tree at once.

Subtree Focus provided the clearest compensation for the small screen. For the 241-node tree, it increased coverage by 64.04 percentage points on the small screen and by 41.70 percentage points on the large screen; the gap between the small and large screens narrowed from 28.22 to 5.88 percentage points. For the 364-node tree, the small-screen increase was 73.92 percentage points and the large-screen increase was 64.53 percentage points, narrowing the inter-screen gap from 18.69 to 9.30 percentage points. Context Collapse provided weaker compensation: the inter-screen gap narrowed to 14.29 percentage points for the 241-node tree and only to 18.18 percentage points for the 364-node tree. This is because Collapse still retains the path and collapsed roots, whereas Focus retains only the current branch and necessary context. The comparison concerns coverage of cards retained by the current condition and does not indicate that the complete tree has become fully visible.

Search moved the fixed target into the viewport in all 15 screen–scale combinations,

showing that reducing screen size did not break target location. Overall, a large screen still displays more absolute context, but the optimisations did not further expand this advantage. Compact and Overview remained stable across screens, Search preserved target location, and Focus substantially reduced the coverage gap between small and large screens. The data therefore support the statement that "display optimisations compensate for some spatial limitations of small screens while retaining the absolute capacity advantage of large screens", rather than that "display optimisations principally increase the advantage of a large screen". This conclusion concerns only card area, context coverage and target location, and is not equivalent to human readability or use efficiency.

7 Conclusion

7.1 Summary

This dissertation implemented a visual behaviour-tree plugin in Godot 4.6 and studied the display problem of large behaviour trees. Godot provides general graph-editing controls but does not provide a complete behaviour-tree editing and execution workflow. Traditional node graphs also occupy a large amount of canvas space as nodes, parameters and debugging information increase.

The final plugin can create, save, load and execute behaviour trees, and supports a blackboard, Decorators, multiple composite-node types, Actions, Conditions, undo, redo and Live Debug. A complex two-dimensional scene uses a 241-node behaviour tree to control enemy detection, defence, melee attack, ranged attack, jumping, climbing, search, retreat, healing and patrol.

Experiments across five tree scales showed that Compact reduced card area by 61.09–61.35%, Overview reduced it by 90.27–90.34%, and Search dimmed 96.15–99.67% of non-target cards. Focus and Collapse also substantially reduced the context currently displayed, and no condition changed saved positions or execution order.

The three-screen experiment showed that complex-tree coverage on the 31.55-inch screen was 2.54 times that on the 15.94-inch screen. The effects of Compact, Overview and Search remained stable across different screens. Overall, the display optimisations improved some objective display and editing-interaction measures for large behaviour trees.

Bibliography

Colledanchise, M. and Ögren, P. (2018). *Behavior Trees in Robotics and AI: An Introduction*. CRC Press. https://doi.org/10.1201/9780429489105.

Furnas, G. W. (1986). Generalized fisheye views. In: *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '86)*, pp. 16–23. https://doi.org/10.1145/22627.22342.

Lamping, J., Rao, R. and Pirolli, P. (1995). A focus+context technique based on hyperbolic geometry for visualizing large hierarchies. In: *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '95)*, pp. 401–408. https://doi.org/10.1145/223904.223956.

Cockburn, A., Karlson, A. and Bederson, B. B. (2008). A review of overview+detail, zooming, and focus+context interfaces. *ACM Computing Surveys*, 41(1), Article 2, pp. 1–31. https://doi.org/10.1145/1456650.1456652.

Song, H., Kim, B., Lee, B. and Seo, J. (2010). A comparative evaluation on tree visualization methods for hierarchical structures with large fan-outs. In: *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI 2010)*, pp. 223–232. https://doi.org/10.1145/1753326.1753359.

Bae, J. and Watson, B. (2011). Developing and evaluating Quilts for the depiction of large layered graphs. *IEEE Transactions on Visualization and Computer Graphics*, 17(12), pp. 2268–2275. https://doi.org/10.1109/TVCG.2011.187.

Dogrusoz, U., Karacelik, A., Safarli, I., Balci, H., Dervishi, L. and Siper, M. C. (2018). Efficient methods and readily customizable libraries for managing complexity of large networks. *PLOS ONE*, 13(5), e0197238. https://doi.org/10.1371/journal.pone.0197238.

Misue, K., Eades, P., Lai, W. and Sugiyama, K. (1995). Layout adjustment and the mental map. *Journal of Visual Languages & Computing*, 6(2), pp. 183–210. https://doi.org/10.1006/jvlc.1995.1010.

Reingold, E. M. and Tilford, J. S. (1981). Tidier drawings of trees. *IEEE Transactions on Software Engineering*, SE-7(2), pp. 223–228. https://doi.org/10.1109/TSE.1981.234519.

Plaisant, C., Grosjean, J. and Bederson, B. B. (2002). SpaceTree: Supporting exploration in large node link tree, design evolution and empirical evaluation. In: *IEEE Symposium on Information Visualization 2002 (INFOVIS 2002)*, pp. 57–64. https://doi.org/10.1109/INFVIS.2002.1173148.

Shneiderman, B. (1996). The eyes have it: A task by data type taxonomy for information visualizations. In: *Proceedings of the 1996 IEEE Symposium on Visual Languages*, pp. 336–343. https://doi.org/10.1109/VL.1996.545307.

Godot Engine contributors (n.d.-a). EditorPlugin. Godot Engine 4.6 documentation. Available at: https://docs.godotengine.org/en/4.6/classes/class_editorplugin.html (Accessed: 24 August 2026).

Godot Engine contributors (n.d.-b). GraphEdit. Godot Engine 4.6 documentation. Available at: https://docs.godotengine.org/en/4.6/classes/class_graphedit.html (Accessed: 24 August 2026).

Epic Games (n.d.). Behavior Tree in Unreal Engine—Overview. Available at: https://dev.epicgames.com/documentation/en-us/unreal-engine/behavior-tree-in-unreal-engine---overview (Accessed: 24 August 2026).